

GiroCerto

Conteúdo completo do projeto — visão geral, mockups de todas as telas e schema do banco de dados.

Gerado em 13/08/2026

Visão geral (README)

GiroCerto

Plataforma de logística de motoboy para lojas locais (restaurantes, açaiterias, padarias etc.), com foco em reduzir o ****ciclo ocioso**** do entregador: tempo de espera na loja + volta vazia após a entrega.

MVP multi-tenant desde o início (via `tenant\_id`), mas sem RLS robusta ainda — cliente único no piloto, RLS entra na Fase 2.

Estrutura

...

db/

 schema.sql # schema Supabase/Postgres — versão mais recente e corrigida

mockups/

 cadastro-loja.html # onboarding do proprietário + dados da loja

 painel-loja.html # painel operacional da loja (pedidos, rotas, integrações)

 app-entregador.html # app do motoboy (cadastro, verificação, rotas)

...

Domínio (principais tabelas)

`tenants` (lojas), `usuarios\_loja` (papéis: dono/operador), `entregadores`,
`horarios\_funcionamento`, `turnos`, `rotas\_entrega`, `tentativas\_despacho`,
`localizacoes\_entregador` (rastreo ao vivo), `alertas\_seguranca`, `pedidos`,
`tentativas\_contato`, `comprovantes\_entrega`, `repasses`, `avaliacoes\_loja`,
`integracoes` (Brendi, WhatsApp Business API, Pix).

Gap conhecido (encontrado ao consolidar as versões)

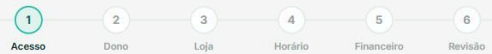
`db/schema.sql` espera que o cadastro do entregador grave
`verificacao\_prazo\_limite` (prazo de 7 dias pra avaliação, calculado pela
aplicação — deixou de ser coluna gerada porque `timestampz + interval` não é
IMMUTABLE pro Postgres, dava erro `42P17` na criação da tabela). O
`mockups/app-entregador.html` atual só grava `verificacao\_enviada\_em` e ainda
não calcula/envia esse prazo — precisa ser ajustado antes de virar código real.

Status

Ainda é só design/schema (mockups HTML estáticos + schema SQL), sem app real rodando. Duas versões dos mockups e do schema circulavam soltas na pasta — consolidadas aqui na versão mais recente/correta de cada arquivo (ver histórico do commit inicial pra detalhes de que mudou entre elas).

Mockups das telas

Cadastro da loja — Criar acesso (passo 1/6)



Criar acesso

É o e-mail e senha que você vai usar pra entrar no painel depois.

E-mail *

Senha *

Use letra maiúscula, número e pelo menos 8 caracteres.

Continuar

Cadastro da loja — Horário de funcionamento (passo 4/6)

 GiroCerto

✓

Acesso

✓

Dono

✓

Loja

4

Horário

5

Financeiro

6

Revisão

Horário de funcionamento

O GiroCerto só despacha motoboy dentro desses horários — mesmo que outra plataforma esteja marcada como aberta.

Domingo

☐

Segunda

☐

Terça

☐

Quarta

☐

Cadastro da loja — Dados bancários / Pix (passo 5/6)



Acesso

Dono

Loja

Horário

5
Financeiro

6
Revisão

Dados bancários

Pra onde o Pix dos pedidos vai cair.

Chave Pix da loja *

O GiroCerto não tem acesso ao seu banco — só uma permissão específica e revogável pra enviar Pix de repasse aos motoboys, quando você configurar isso com seu provedor de pagamento. Você pode revogar quando quiser, direto no app do seu banco.

Voltar

Revisar cadastro

App do entregador — Cadastro do motoboy

 GiroCerto

Cadastro do entregador

Leva uns minutos — documentos entram em avaliação (até 7 dias).

 Moto

 Bicicleta

Nome completo

E-mail

Senha

Data de nascimento

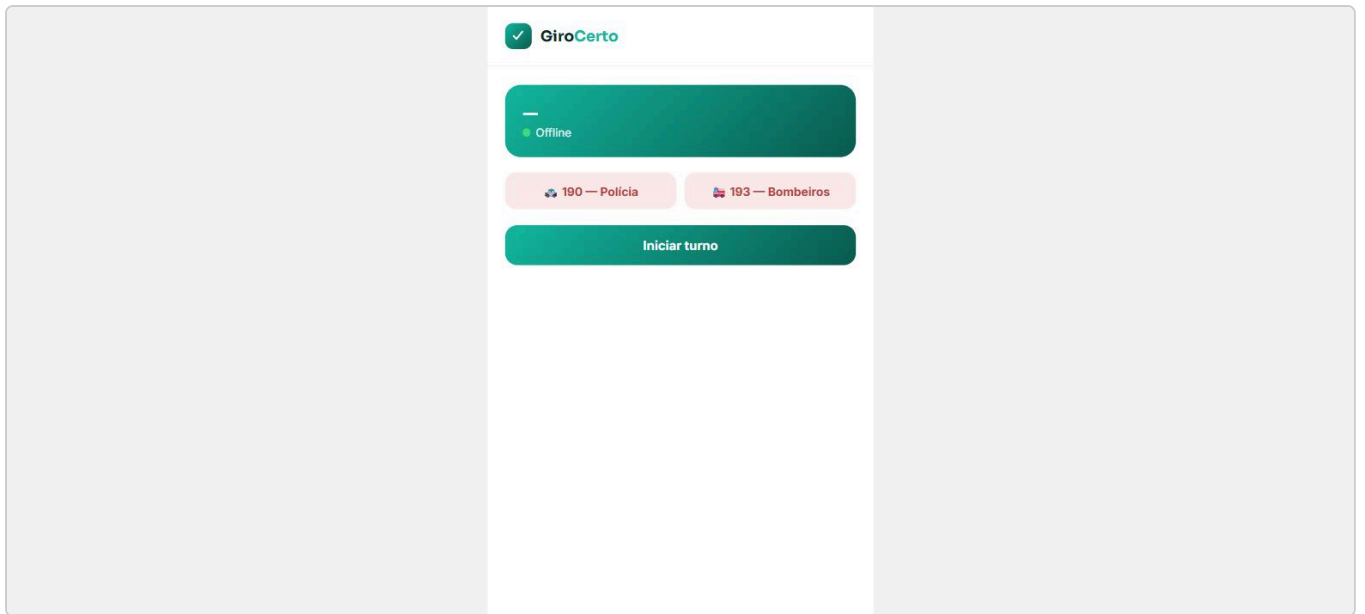
dd/mm/aaaa

Endereço

Número

CEP

App do entregador — Início de turno



Painel da loja — Visão operacional

 GiroCerto

Loja Exemplo

Pedidos

Painel operacional

Entregadores

Relatórios

Integrações

Painel operacional

Pedidos hoje

Rotas ativas

Motoboy's ativos

Rotas ativas

Carregando...

Motoboy's hoje

Carregando...

Schema do banco de dados (db/schema.sql)

```
-- =====
-- GiroCerto – Schema inicial (Supabase / Postgres)
-- MVP: foco em ciclo ocioso (espera na loja + volta vazia)
-- Multi-tenant desde o início via tenant_id, mas sem RLS
-- robusta ainda – cliente único no piloto, RLS entra na Fase 2.
-- =====

create extension if not exists pgcrypto; -- necessário para gen_random_uuid()

-- -----
-- TENANTS: cada cliente (rede de restaurante, empresa de motofrete)
-- -----

create table if not exists tenants (
  id uuid primary key default gen_random_uuid(),
  nome text not null, -- nome da loja

  -- -----
  -- CADASTRO (seção 25): dados do proprietário + dados da loja
  -- -----
  -- proprietário
  proprietario_nome text,
  proprietario_cpf text,
  proprietario_data_nascimento date,
  proprietario_endereco text,
  proprietario_numero_endereco text,
  proprietario_cep text,
  proprietario_documento_foto_url text, -- RG ou CNH do proprietário

  -- loja (negócio)
  cnpj text,
  cnpj_documento_foto_url text, -- cartão CNPJ ou contrato social
  endereco_loja text,
  numero_loja text,
  cep_loja text,

  -- segmento (seção 33) – alimenta um ponto de partida razoável de tempo de
  -- preparo antes de acumular histórico próprio (seção 32)
  segmento text
  check (segmento in ('acai_sorveteria', 'padaria', 'hamburgueria', 'pizzaria',
    'salgaderia', 'lanchonete', 'doceria_bolos', 'sushi_japonesa', 'outro')),
  tempo_preparo_padrao_min integer, -- pré-preenche o campo por pedido; cozinha pode sobrescrever

  -- parâmetros de tarifa do motoboy – calibrados para São Paulo (capital);
  -- cada tenant pode ajustar os seus se operar em outra cidade
  tarifa_minima numeric(10,2) not null default 10.00, -- valor mínimo por rota, já cobre o km_minimo_incluso
  km_minimo_incluso numeric(5,2) not null default 2.00, -- km cobertos pela tarifa mínima, sem custo adicional
  valor_por_km_adicional numeric(10,2) not null default 2.00, -- cobrado só sobre o que exceder o km_minimo_incluso

  -- raios de despacho (seção 33) – dois conceitos diferentes:
  -- 1) distância pra CHAMAR o motoboy até a loja (além disso não compensa pra ele)
  -- 2) distância máxima confortável da ENTREGA em si, da loja até o cliente
  raio_chamada_motoboy_km numeric(4,1) not null default 1.5,
  raio_maximo_entrega_km numeric(4,1) not null default 6.0,

  -- LGPD (seção 37) – carimbo de quando o proprietário aceitou os termos
  -- de tratamento de dados; sem isso não tem como provar consentimento
  consentimento_lgpd_aceito_em timestamptz,

  -- % do valor do pedido reembolsado ao cliente quando ele não é localizado
  -- após o protocolo de contato (ver tentativas_contato) se esgotar
  percentual_reembolso_sem_contato numeric(5,2) not null default 50.00,
```

```

-- tempo de espera do motoboy (loja ou endereço do cliente): tolerado sem custo
-- até um limite, remunerado à parte a partir daí – ataca a dor de "espera não paga"
tempo_espera_tolerado_min integer not null default 10,
valor_por_minuto_espera_excedente numeric(10,2) not null default 0.50,

-- infraestrutura de apoio oferecida ao entregador – vira critério de reputação
-- da loja entre os motoboys, não só operacional
oferece_banheiro boolean not null default true,
oferece_abrigo_chuva boolean not null default true,

-- chave Pix da loja: usada pra gerar o QR Code dinâmico cobrado do cliente
-- na entrega (seção 15) – o pagamento cai direto aqui, sem intermediário
chave_pix text,

-- frequência do repasse automatizado ao motoboy (seção 16): por entrega
-- (paga segundos após cada entrega) ou em lote no fim do turno
frequencia_repasse text not null default 'por_entrega'
    check (frequencia_repasse in ('por_entrega', 'fim_de_turno')),

-- fadiga (seção 19): a partir de quantas horas ativas seguidas sem pausa o
-- sistema alerta ao finalizar, e por quanto tempo bloqueia reativação se
-- ele finalizar mesmo assim
horas_alerta_fadiga numeric(4,1) not null default 8.0,
horas_descanso_obrigatorio numeric(4,1) not null default 8.0,

-- repique da notificação de despacho (seção 23): repete o mesmo toque calmo
-- a cada N segundos até aceitar ou até o timeout, quando falha automático
-- pro próximo entregador disponível (redundância, seção 12)
segundos_repique_notificacao integer not null default 8,
segundos_timeout_despacho integer not null default 24,

-- detecção de desvio de rota (seção 26) – nunca aciona 190 sozinho;
-- só depois de aguardar confirmação do próprio entregador
km_desvio_alerta numeric(5,2) not null default 3.0,
segundos_para_confirmar_seguranca integer not null default 90,

-- ajuste de tempo de preparo (seção 31.3) – multiplicador aplicado sobre
-- tempo_preparo_estimado_min ao calcular pronto_previsto_em, atualizado
-- periodicamente a partir do histórico real (pedidos.pronto_em vs
-- pedidos.tempo_preparo_estimado_min). 1.00 = sem ajuste (padrão até
-- acumular dados suficientes); >1 = cozinha costuma subestimar
fator_ajuste_preparo numeric(4,2) not null default 1.00,
ajuste_preparo_atualizado_em timestamptz,

    criado_em timestamptz not null default now()
);

-- -----
-- ENTREGADORES: motoboys de cada tenant
-- -----
create table if not exists entregadores (
    id uuid primary key default gen_random_uuid(),
    tenant_id uuid not null references tenants(id) on delete cascade,
    nome text not null,
    telefone text,
    status text not null default 'offline'
        check (status in ('offline', 'disponivel', 'pausado', 'a_caminho_da_loja', 'em_rota', 'na_loja')),
    lat double precision,
    lng double precision,
    localizacao_atualizada_em timestamptz,
    possui_maquinhinha boolean not null default false, -- necessário pra rotear pedidos pagos no cartão

-- vínculo: motoboy fixo (custo já pago, independe do volume) x freelance (pago por entrega)
tipo_vinculo text not null default 'freelance'

```

```

    check (tipo_vinculo in ('fixo', 'freelance')),
valor_fixo numeric(10,2),          -- só se tipo_vinculo = 'fixo': valor da diária ou mensalidade
periodicidade_fixo text
    check (periodicidade_fixo in ('diaria', 'mensal')),

-- chave Pix do motoboy: usada pro repasse em lote no fim do turno (seção 16)
chave_pix text,

-- bloqueio temporário de descanso obrigatório (seção 19) – impede iniciar
-- novo turno antes desse horário; null = sem bloqueio ativo
bloqueado_ate timestampz,

-- pausar durante uma entrega em andamento (seção 33): não interrompe a
-- rota atual, só impede receber a próxima assim que essa terminar
pausar_apos_rota_atual boolean not null default false,

-- LGPD (seção 37/38)
consentimento_lgpd_aceito_em timestampz,
dados_anonizados_em timestampz, -- preenchido quando ele pede exclusão (ver seção 38);
                                -- CPF/RG/endereço/fotos são apagados, mas repasses e
                                -- entregas continuam existindo pra fechar a contabilidade

-- app externo usado pra navegação (seção 21) – GiroCerto traça a rota
-- (OSRM) mas a navegação turn-by-turn com radar/acidente/obra é feita
-- por deep link pro app que ele já usa e confia
app_navegacao_preferido text not null default 'waze'
    check (app_navegacao_preferido in ('waze', 'google_maps')),

-- -----
-- DOCUMENTAÇÃO E VERIFICAÇÃO (seção 24)
-- -----
tipo_veiculo text not null default 'moto'
    check (tipo_veiculo in ('moto', 'bicicleta')),
data_nascimento date, -- usado só pra checar faixa etária (15-18 = só bicicleta)

-- identidade e endereço
cpf text,          -- moto (vinculado à CNH)
rg_numero text,    -- bicicleta
endereco text,
numero_residencia text,
cep text,

-- documentos moto
cnh_numero text,
cnh_validade date,
cnh_foto_url text,
crlv_validade date,          -- documento da moto (CRLV)
crlv_foto_url text,
placa text,
comprovante_residencia_foto_url text,

-- controle de aviso de vencimento (seção 33) – evita mandar a mesma
-- mensagem repetida; reseta quando o documento é atualizado
cnh_alerta_enviado_em timestampz,
crlv_alerta_enviado_em timestampz,

-- documentos bicicleta (15-18 anos)
foto_rg_url text,
foto_rg_segurando_url text, -- selfie segurando o RG, confere identidade
foto_bicicleta_url text,
responsavel_nome text,          -- exigido se menor de 18 (ver nota na seção 24)
responsavel_documento_foto_url text,

-- status da verificação – o "reprovado" automático (documento vencido)
-- acontece na hora; o "aprovado"/"reprovado" manual tem prazo de até 7 dias

```

```

status_verificacao text not null default 'em_avaliacao'
check (status_verificacao in ('em_avaliacao', 'aprovado', 'reprovado')),
motivo_reprovacao text
check (motivo_reprovacao in ('cnh_vencida', 'crlv_vencido', 'documento_ilegivel', 'informacao_divergente',
'outro')),
verificacao_enviada_em timestampz,
-- prazo de avaliação (seção 24): antes era coluna gerada, mas
-- "timestampz + interval" não é imutável pro Postgres (depende do fuso
-- da sessão) – erro 42P17 ao criar a tabela. Agora é preenchido pela
-- aplicação no momento do cadastro (ver app-entregador.html):
-- verificacao_prazo_limite = verificacao_enviada_em + 7 dias
verificacao_prazo_limite timestampz,
aprovado_por uuid, -- referencia usuarios_loja(id); FK adicionada após a criação dessa tabela, mais abaixo
aprovado_em timestampz,

-- -----
-- ACESSO AO SISTEMA (seção 26) – a senha em si NUNCA fica nesta
-- tabela: o Supabase Auth já cuida do hash/força da senha; aqui
-- só guardamos o vínculo e o identificador de login alternativo
-- -----
auth_user_id uuid references auth.users(id),
email text, -- login por e-mail (alternativa ao CPF)

criado_em timestampz not null default now()
);

create unique index if not exists idx_entregadores_email
on entregadores (email) where email is not null;

create unique index if not exists idx_entregadores_auth_user
on entregadores (auth_user_id) where auth_user_id is not null;

create index if not exists idx_entregadores_status_verificacao
on entregadores (tenant_id, status_verificacao);

create index if not exists idx_entregadores_tenant_status
on entregadores (tenant_id, status);

-- -----
-- USUARIOS_LOJA: quem faz login no painel do lojista (dono e/ou
-- funcionários do balcão) – faltava esse vínculo até agora; sem
-- ele não dá pra saber "de qual loja é essa pessoa logada" pra
-- aplicar RLS (seção 37) de forma nenhuma
-- -----
create table if not exists usuarios_loja (
id uuid primary key default gen_random_uuid(),
tenant_id uuid not null references tenants(id) on delete cascade,
auth_user_id uuid not null references auth.users(id),
nome text not null,
papel text not null default 'funcionario'
check (papel in ('dono', 'funcionario')),

-- PIN de segurança pra aba de Integrações (seção 40) – nunca guardado em
-- texto puro, sempre hash via pgcrypto (crypt/gen_salt); definido e
-- verificado só pelas funções abaixo, nunca por update direto na tabela
pin_integracoes_hash text,

criado_em timestampz not null default now()
);

create unique index if not exists idx_usuarios_loja_auth_user
on usuarios_loja (auth_user_id);

create index if not exists idx_usuarios_loja_tenant
on usuarios_loja (tenant_id);

```

```

alter table entregadores
  add constraint fk_entregadores_aprovado_por
  foreign key (aprovado_por) references usuarios_loja(id);

-- -----
-- HORARIOS_FUNCIONAMENTO: dias e períodos em que a loja realmente
-- opera (seção 33) – o motor de despacho só chama motoboy dentro
-- desses períodos, resolvendo o "motoboy chegou e a loja tava
-- fechada" (esqueceram de fechar em outra plataforma, etc.)
-- -----
create table if not exists horarios_funcionamento (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references tenants(id) on delete cascade,
  dia_semana smallint not null check (dia_semana between 0 and 6), -- 0=domingo ... 6=sábado
  periodo_inicio time not null,
  periodo_fim time not null,
  criado_em timestamptz not null default now()
);

create index if not exists idx_horarios_funcionamento_tenant
  on horarios_funcionamento (tenant_id, dia_semana);

-- -----
-- TURNOS: sessão de trabalho do motoboy – do "iniciar turno" ao
-- "finalizar turno" (seção 17). Agrupa as entregas e dispara o
-- repasse acumulado quando ele encerra, se frequencia_repasse
-- do tenant for 'fim_de_turno'.
-- -----
create table if not exists turnos (
  id uuid primary key default gen_random_uuid(),
  entregador_id uuid not null references entregadores(id) on delete cascade,
  status text not null default 'ativo'
  check (status in ('ativo', 'finalizado')),
  iniciado_em timestamptz not null default now(),
  finalizado_em timestamptz,
  teve_pausa boolean not null default false, -- true assim que ele pausar ao menos 1 vez nesse turno
  ultimo_checkin_bemestar timestamptz not null default now(), -- seção 20: reinicia a cada 1h de turno ativo
  total_entregas integer, -- calculado ao finalizar
  total_repasseado numeric(10,2), -- calculado ao finalizar, soma do que foi pago nesse turno
  criado_em timestamptz not null default now()
);

create index if not exists idx_turnos_entregador_status
  on turnos (entregador_id, status);

-- -----
-- ROTAS_ENTREGA: agrupamento de pedidos numa mesma saída do motoboy
-- (criada antes de PEDIDOS porque pedidos referencia rota_id)
-- -----
create table if not exists rotas_entrega (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references tenants(id) on delete cascade,
  entregador_id uuid references entregadores(id),

  status text not null default 'planejada'
  check (status in ('planejada', 'a_caminho_da_loja', 'em_entrega', 'concluida', 'cancelada')),

  tempo_total_estimado_min integer,
  despachado_em timestamptz, -- momento em que o motoboy foi efetivamente chamado
  iniciada_em timestamptz, -- motoboy saiu da loja com os pedidos
  concluida_em timestamptz,

  -- código de retirada (seção 24): loja e motoboy veem o mesmo número;
  -- ele fala esse código na loja pra confirmar que é quem deveria pegar

```

```

codigo_retirada text not null default lpad(floor(random() * 10000)::text, 4, '0'),

-- traçado planejado (polyline do OSRM) – necessário pra detectar desvio
-- de rota (seção 26); sem isso não dá pra saber "quanto ele se afastou"
rota_polyline text,

criado_em timestampz not null default now()
);

create index if not exists idx_rotas_tenant_status
on rotas_entrega (tenant_id, status);

-- -----
-- TENTATIVAS_DESPACHO: cada vez que um entregador é chamado pra
-- uma rota – registra repique/timeout e viabiliza o failover
-- automático pro próximo disponível (seção 23)
-- -----
create table if not exists tentativas_despacho (
  id uuid primary key default gen_random_uuid(),
  rota_id uuid not null references rotas_entrega(id) on delete cascade,
  entregador_id uuid not null references entregadores(id),
  notificado_em timestampz not null default now(),
  resultado text
  check (resultado in ('aceito', 'recusado', 'sem_resposta')),
  respondido_em timestampz
);

create index if not exists idx_tentativas_despacho_rota
on tentativas_despacho (rota_id);

-- -----
-- LOCALIZACOES_ENTREGADOR: histórico de posição (não só a atual),
-- alimenta o detector de desvio de rota e serve de log bruto de
-- telemetria (princípio da seção 12 – decisão de despacho não é
-- a única coisa que vale registrar, o trajeto real também)
-- -----
create table if not exists localizacoes_entregador (
  id uuid primary key default gen_random_uuid(),
  entregador_id uuid not null references entregadores(id) on delete cascade,
  rota_id uuid references rotas_entrega(id), -- null se não estiver em rota
  lat double precision not null,
  lng double precision not null,
  registrado_em timestampz not null default now()
);

create index if not exists idx_localizacoes_entregador_tempo
on localizacoes_entregador (entregador_id, registrado_em);

create index if not exists idx_localizacoes_rota
on localizacoes_entregador (rota_id);

-- -----
-- ALERTAS_SEGURANCA: desvio de rota além do limite ou SOS manual
-- (seção 26) – fluxo de confirmação humana antes de qualquer
-- escalonamento, nunca aciona 190 sozinho e em silêncio
-- -----
create table if not exists alertas_seguranca (
  id uuid primary key default gen_random_uuid(),
  entregador_id uuid not null references entregadores(id) on delete cascade,
  rota_id uuid references rotas_entrega(id),
  tipo text not null check (tipo in ('desvio_rota', 'sos_manual')),
  distancia_desvio_km numeric(6,2),
  status text not null default 'aguardando_confirmacao'
  check (status in ('aguardando_confirmacao', 'confirmado_ok', 'escalado_loja', 'acionado_190', 'falso_alarme')),
  criado_em timestampz not null default now(),

```

```

    resolvido_em timestampz
);

create index if not exists idx_alertas_seguranca_entregador
on alertas_seguranca (entregador_id, status);

-- -----
-- PEDIDOS: cada pedido a ser entregue
-- -----

create table if not exists pedidos (
    id uuid primary key default gen_random_uuid(),
    tenant_id uuid not null references tenants(id) on delete cascade,

    -- de onde o pedido veio (GiroCerto não tem cardápio próprio – recebe de fora)
    origem text not null default 'manual'
    check (origem in ('manual', 'whatsapp', 'ifood', 'cardapio_proprio')),
    cliente_nome text,
    cliente_telefone text,    -- usado pro motoboy contatar e pra enviar o link de rastreo

    -- código de entrega (seção 24): enviado ao cliente por WhatsApp junto com o
    -- aviso "a caminho"; o cliente fala esse código pro motoboy na porta
    codigo_entrega text not null default lpad(floor(random() * 10000)::text, 4, '0'),

    -- endereço de entrega
    endereco text not null,
    lat double precision,
    lng double precision,

    -- ciclo de vida do pedido, incluindo o status de preparo na cozinha
    status text not null default 'recebido'
    check (status in (
        'recebido',      -- pedido entrou no sistema
        'em_preparo',    -- cozinha começou a preparar
        'pronto',        -- cozinha finalizou
        'a_caminho',     -- motoboy pegou e está entregando
        'entregue',      -- concluído
        'cancelado'
    )),

    -- campos centrais para o despacho no tempo certo
    tempo_preparo_estimado_min integer,    -- input manual da cozinha, ex: 12
    pronto_previsto_em timestampz,         -- criado_em + tempo_preparo_estimado_min (recalculável)
    tempo_deslocamento_loja_min integer,   -- calculado via OSRM (distância motoboy -> loja)

    -- roteirização
    rota_id uuid references rotas_entrega(id),
    ordem_na_rota integer,                 -- posição da parada dentro da rota

    -- financeiro: valor do pedido (o que o cliente paga) e valor de entrega (o que vai pro motoboy)
    valor_pedido numeric(10,2) not null default 0,
    valor_entrega numeric(10,2),

    -- forma de pagamento – v1 só usa 'pix' (pago_antecipado=true = online antes da
    -- saída; pago_antecipado=false = QR Code dinâmico na entrega, seção 15/16).
    -- 'cartao' e 'dinheiro' ficam reservados no schema pra Fase 2 (não oferecidos
    -- na UI do v1) – assim uma eventual reativação não exige migração de banco.
    forma_pagamento text not null default 'pix'
    check (forma_pagamento in ('pix', 'cartao', 'dinheiro')),
    pago_antecipado boolean not null default false, -- true = Pix já pago antes da saída, nada a cobrar na entrega
    troco_para numeric(10,2),                      -- reservado pra Fase 2 (dinheiro) – não usado no v1
    valor_troco numeric(10,2)
    generated always as (
        case when troco_para is not null then troco_para - valor_pedido else null end
    ) stored,                                     -- calculado automaticamente: quanto devolver de troco

```

```

-- confirmação de pagamento – serve tanto pro Pix antecipado quanto pro Pix
-- na entrega via QR Code dinâmico (ver seção 14); pix_txid é o identificador
-- da cobrança no provedor (Mercado Pago/Stone/etc.), útil pra conciliação
-- pago boolean not null default false,
-- pago_em timestampz,
-- pix_txid text,

-- criado_em timestampz not null default now(),
-- pronto_em timestampz,
-- entregue_em timestampz,

-- protocolo de "não encontrei o cliente": true enquanto as tentativas de
-- contato (ver tentativas_contato) ainda não resolveram a entrega
-- contato_pendente boolean not null default false,

-- resolução quando o protocolo de contato se esgota sem sucesso
-- motivo_cancelamento text
-- check (motivo_cancelamento in ('cliente_nao_localizado')),
-- valor_reembolsado numeric(10,2), -- calculado a partir de tenants.percentual_reembolso_sem_contato
-- item_retorna_loja boolean not null default true, -- false = motoboy liberado, não precisa voltar com o pedido

-- avaliação da ENTREGA (não do pedido/comida) – coletada pelo link de rastreo (seção 7),
-- serve pra separar reclamação de comida (não é conosco) de reclamação de entrega (é conosco)
-- avaliacao_entrega smallint check (avaliacao_entrega between 1 and 5),
-- avaliacao_comentario text,

-- tipo de local (afeta tempo estimado de parada) e tempo de espera medido –
-- usado pra calcular o bônus de espera excedente (ver tenants)
-- tipo_local text not null default 'casa'
-- check (tipo_local in ('casa', 'apartamento', 'comercial')),
-- tempo_espera_min integer -- medido pelo app: entre chegada no local e conclusão da parada
);

create index if not exists idx_pedidos_tenant_status
on pedidos (tenant_id, status);

create index if not exists idx_pedidos_rota
on pedidos (rota_id);

create index if not exists idx_pedidos_tenant_forma_pagamento
on pedidos (tenant_id, forma_pagamento);

-- -----
-- TENTATIVAS_CONTATO: histórico de ligações/mensagens quando o
-- motoboy não encontra o cliente no endereço
-- -----
create table if not exists tentativas_contato (
    id uuid primary key default gen_random_uuid(),
    pedido_id uuid not null references pedidos(id) on delete cascade,
    tipo text not null check (tipo in ('ligacao', 'mensagem')),
    resultado text check (resultado in ('atendeu', 'nao_atendeu', 'sem_resposta', 'respondeu')),
    criado_em timestampz not null default now()
);

create index if not exists idx_tentativas_pedido
on tentativas_contato (pedido_id);

-- -----
-- COMPROVANTES_ENTREGA: prova de entrega (foto + geolocalização)
-- -----
create table if not exists comprovantes_entrega (
    id uuid primary key default gen_random_uuid(),
    pedido_id uuid not null references pedidos(id) on delete cascade,
    foto_url text,
    lat double precision,

```

```

    lng double precision,
    codigo_confirmado boolean not null default false, -- true = cliente informou o código certo (seção 24)
    criado_em timestamptz not null default now()
);

-- -----
-- REPASSES: valor devido a cada entregador por pedido entregue,
-- pago via API de Pix do provedor da loja (seção 16) – status muda
-- pra 'pago' automaticamente quando a transferência é confirmada,
-- sem ação manual da loja por entregador
-- -----
create table if not exists repasses (
    id uuid primary key default gen_random_uuid(),
    entregador_id uuid not null references entregadores(id) on delete cascade,
    pedido_id uuid not null references pedidos(id) on delete cascade,
    turno_id uuid references turnos(id), -- turno em que a entrega ocorreu (seção 17)
    valor numeric(10,2) not null,
    status text not null default 'pendente'
    check (status in ('pendente', 'pago')),
    pix_txid text, -- identificador da transferência Pix enviada (confirmação da API)
    pago_em timestamptz,
    criado_em timestamptz not null default now()
);

create index if not exists idx_repasses_entregador_status
    on repasses (entregador_id, status);

-- -----
-- AVALIACOES_LOJA: o motoboy avalia a loja ao finalizar o turno
-- (seção 31.2) – base do Selo Entrega Justa. Diferente de
-- oferece_banheiro/oferece_abrigo_chuva (autodeclarado), isso é
-- feedback real de quem trabalhou lá, não dá pra forjar sozinho.
-- -----
create table if not exists avaliacoes_loja (
    id uuid primary key default gen_random_uuid(),
    tenant_id uuid not null references tenants(id) on delete cascade,
    entregador_id uuid not null references entregadores(id) on delete cascade,
    turno_id uuid references turnos(id),
    nota smallint not null check (nota between 1 and 5),
    comentario text,
    criado_em timestamptz not null default now()
);

create index if not exists idx_avaliacoes_loja_tenant
    on avaliacoes_loja (tenant_id, criado_em);

-- -----
-- SELO_ENTREGA_JUSTA: view calculada em tempo real, não um campo
-- que alguém marca manualmente – exige infraestrutura declarada
-- (banheiro/abrigo) E avaliação real dos motoboys nos últimos 30
-- dias, com volume mínimo pra não dar pra manipular com 2 notas.
-- -----
create or replace view selo_entrega_justa as
select
    t.id as tenant_id,
    t.nome,
    t.oferece_banheiro,
    t.oferece_abrigo_chuva,
    coalesce(avg(al.nota), 0)::numeric(3,2) as media_avaliacao_motoboys,
    count(al.id) as total_avaliacoes_30d,
    (
        t.oferece_banheiro
        and t.oferece_abrigo_chuva
        and coalesce(avg(al.nota), 0) >= 4.0
        and count(al.id) >= 10
    )

```

```

    ) as selo_ativo
from tenants t
left join avaliacoes_loja al
    on al.tenant_id = t.id
    and al.criado_em > now() - interval '30 days'
group by t.id, t.nome, t.oferece_banheiro, t.oferece_abrigo_chuva;

-- =====
-- ROW LEVEL SECURITY (seções 37/38)
-- IMPORTANTE: o backend (Node.js, seção 4) usa a service role key
-- do Supabase pra rodar o motor de despacho, o repasse automatizado
-- e os jobs periódicos – a service role IGNORA RLS por definição.
-- Essas políticas protegem o acesso direto do cliente (app do
-- motoboy, painel do lojista falando com o Supabase), não o backend.
-- =====

alter table pedidos enable row level security;
alter table rotas_entrega enable row level security;
alter table entregadores enable row level security;
alter table repasses enable row level security;
alter table tenants enable row level security;
alter table usuarios_loja enable row level security;
alter table comprovantes_entrega enable row level security;
alter table tentativas_contato enable row level security;
alter table turnos enable row level security;
alter table avaliacoes_loja enable row level security;
alter table alertas_seguranca enable row level security;
alter table localizacoes_entregador enable row level security;
alter table horarios_funcionamento enable row level security;

-- pedidos: loja vê e cria os do seu tenant
create policy "loja ve seus pedidos" on pedidos for select using (
    tenant_id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));
create policy "loja cria pedidos no seu tenant" on pedidos for insert with check (
    tenant_id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));
create policy "loja atualiza seus pedidos" on pedidos for update using (
    tenant_id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));

-- rotas_entrega: loja e o entregador designado
create policy "loja ve suas rotas" on rotas_entrega for select using (
    tenant_id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));
create policy "entregador ve suas proprias rotas" on rotas_entrega for select using (
    entregador_id in (select id from entregadores where auth_user_id = auth.uid()));
create policy "entregador atualiza suas proprias rotas" on rotas_entrega for update using (
    entregador_id in (select id from entregadores where auth_user_id = auth.uid()));

-- entregadores: cada um vê e edita o próprio cadastro; a loja só vê (aprovação passa pelo backend)
create policy "entregador ve e edita seu cadastro" on entregadores for select using (
    auth_user_id = auth.uid());
create policy "entregador cria seu proprio cadastro" on entregadores for insert with check (
    auth_user_id = auth.uid());
create policy "entregador atualiza seu proprio cadastro" on entregadores for update using (
    auth_user_id = auth.uid());
create policy "loja ve entregadores do seu tenant" on entregadores for select using (
    tenant_id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));

-- repasses: só o próprio entregador vê os seus
create policy "entregador ve seus proprios repasses" on repasses for select using (
    entregador_id in (select id from entregadores where auth_user_id = auth.uid()));

-- tenants: qualquer usuário autenticado pode criar um tenant novo no cadastro
-- (ainda não existe usuarios_loja vinculado nesse momento – o vínculo vem
-- logo em seguida); ver/editar depois disso já exige pertencer ao tenant
create policy "usuario autenticado cria um tenant" on tenants for insert with check (
    auth.uid() is not null);

```

```

create policy "loja ve e edita seu proprio tenant" on tenants for select using (
    id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));
create policy "loja atualiza seu proprio tenant" on tenants for update using (
    id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));

-- usuarios_loja: só enxerga colegas do mesmo tenant, e só cria o próprio vínculo
create policy "usuario ve colegas do mesmo tenant" on usuarios_loja for select using (
    tenant_id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));
create policy "usuario cria seu proprio vinculo" on usuarios_loja for insert with check (
    auth_user_id = auth.uid());

-- comprovantes_entrega: loja do pedido + o entregador que entregou
create policy "loja ve comprovantes dos seus pedidos" on comprovantes_entrega for select using (
    pedido_id in (select id from pedidos where tenant_id in
        (select tenant_id from usuarios_loja where auth_user_id = auth.uid())));
create policy "entregador cria comprovante da propria entrega" on comprovantes_entrega for insert with check (
    pedido_id in (select p.id from pedidos p join rotas_entrega r on r.id = p.rota_id
        where r.entregador_id in (select id from entregadores where auth_user_id = auth.uid())));

-- tentativas_contato: só a loja do pedido em questão
create policy "loja ve tentativas de contato dos seus pedidos" on tentativas_contato for select using (
    pedido_id in (select id from pedidos where tenant_id in
        (select tenant_id from usuarios_loja where auth_user_id = auth.uid())));

-- turnos: o próprio entregador e a loja onde ele atua
create policy "entregador ve e edita seus proprios turnos" on turnos for all using (
    entregador_id in (select id from entregadores where auth_user_id = auth.uid()));
create policy "loja ve turnos dos seus entregadores" on turnos for select using (
    entregador_id in (select id from entregadores where tenant_id in
        (select tenant_id from usuarios_loja where auth_user_id = auth.uid())));

-- avaliacoes_loja: entregador cria a sua, loja só lê o agregado (não quem disse o quê)
create policy "entregador cria avaliacao da loja" on avaliacoes_loja for insert with check (
    entregador_id in (select id from entregadores where auth_user_id = auth.uid()));

-- alertas_seguranca: bem restrito – só o próprio entregador e a loja envolvida
create policy "entregador ve e atualiza seus alertas" on alertas_seguranca for all using (
    entregador_id in (select id from entregadores where auth_user_id = auth.uid()));
create policy "loja ve alertas dos seus entregadores" on alertas_seguranca for select using (
    entregador_id in (select id from entregadores where tenant_id in
        (select tenant_id from usuarios_loja where auth_user_id = auth.uid())));

-- localizacoes_entregador: dado sensível – só o próprio e a loja, nunca público
create policy "entregador gerencia sua propria localizacao" on localizacoes_entregador for all using (
    entregador_id in (select id from entregadores where auth_user_id = auth.uid()));
create policy "loja ve localizacao dos seus entregadores" on localizacoes_entregador for select using (
    entregador_id in (select id from entregadores where tenant_id in
        (select tenant_id from usuarios_loja where auth_user_id = auth.uid())));

-- horarios_funcionamento: público pra leitura (o app precisa saber se está aberto
-- antes mesmo de qualquer login), só a loja edita o próprio
create policy "qualquer um le horario de funcionamento" on horarios_funcionamento for select using (true);
create policy "loja edita seu proprio horario" on horarios_funcionamento for all using (
    tenant_id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid()));

-- =====
-- STORAGE (seção 39) – buckets privados pros documentos e fotos
-- de entrega que as telas de cadastro e entrega precisam. Rode
-- isso depois de tudo acima; os buckets precisam existir antes
-- de qualquer upload funcionar.
-- =====

insert into storage.buckets (id, name, public)
values
    ('documentos-privados', 'documentos-privados', false),

```

```

('comprovantes', 'comprovantes', false)
on conflict (id) do nothing;

-- só o próprio usuário autenticado sobe arquivo na pasta que ele criou
create policy "usuario sobe seus proprios documentos" on storage.objects
  for insert with check (
    bucket_id = 'documentos-privados' and auth.uid() is not null
  );

create policy "usuario ve seus proprios documentos" on storage.objects
  for select using (
    bucket_id = 'documentos-privados' and auth.uid() is not null
  );

create policy "entregador sobe foto de entrega" on storage.objects
  for insert with check (
    bucket_id = 'comprovantes' and auth.uid() is not null
  );

create policy "loja ve fotos de entrega dos seus pedidos" on storage.objects
  for select using (
    bucket_id = 'comprovantes' and auth.uid() is not null
  );

-- =====
-- INTEGRACOES (seção 40) – credenciais de API por loja (Brendi,
-- WhatsApp, provedor de Pix). Separada de `tenants` de propósito:
-- é dado mais sensível, acessado com menos frequência, e só o
-- dono (não funcionário) pode ver ou editar.
-- =====
create table if not exists integracoes (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references tenants(id) on delete cascade,

  brendi_api_key text,

  whatsapp_phone_number_id text,
  whatsapp_access_token text,
  whatsapp_webhook_verify_token text,

  pix_provider text check (pix_provider in ('mercado_pago', 'asaas', 'stone', 'outro')),
  pix_provider_api_key text,

  atualizado_em timestamptz not null default now()
);

create unique index if not exists idx_integracoes_tenant
  on integracoes (tenant_id);

alter table integracoes enable row level security;

-- só o dono (não funcionário) vê ou edita – funcionário não tem acesso nenhum,
-- nem pedindo PIN, porque a checagem de papel já bloqueia antes disso
create policy "dono ve e edita integracoes do seu tenant" on integracoes for all using (
  tenant_id in (select tenant_id from usuarios_loja where auth_user_id = auth.uid() and papel = 'dono')
);

-- =====
-- PIN de segurança da aba de Integrações – definido e verificado
-- só por essas duas funções; a coluna pin_integracoes_hash nunca
-- é lida nem escrita diretamente pelo cliente (sem policy de
-- select/update nela na prática, só via RPC abaixo)
-- =====
create or replace function set_pin_integracoes(novo_pin text)
returns void

```

```

language sql
security definer
as $$
    update usuarios_loja
    set pin_integracoes_hash = crypt(novo_pin, gen_salt('bf'))
    where auth_user_id = auth.uid() and papel = 'dono';
$$;

create or replace function verificar_pin_integracoes(tentativa text)
returns boolean
language sql
security definer
as $$
    select coalesce(
        (select pin_integracoes_hash = crypt(tentativa, pin_integracoes_hash)
         from usuarios_loja
         where auth_user_id = auth.uid() and papel = 'dono' and pin_integracoes_hash is not null),
        false
    );
$$;

create or replace function tem_pin_integracoes()
returns boolean
language sql
security definer
as $$
    select coalesce(
        (select pin_integracoes_hash is not null
         from usuarios_loja where auth_user_id = auth.uid() and papel = 'dono'),
        false
    );
$$;

```